TIA Portal V18 + PLCSIM Advanced V5.0 — OPC UA bring-up runbook

Goal: stand up a **simulated Siemens S7-1500** whose native **OPC UA server** we can read (monitor PLC → software) and write (command software → PLC) from our existing app — **with no app code change**, exactly like we proved with OpenPLC. Only the `.env` (endpoint + NodeIds + security) changes.

> Why this works with zero code: our backend is a **generic OPC UA client** (`backend/src/services/plc/`, `node-opcua`). OpenPLC v4 and the Siemens S7-1500 both expose native OPC-UA servers, so switching between them is a **config change only**. See CLAUDE.md.

Verified installed on this PC: TIA Portal V18 SP1 · S7-PLCSIM **Advanced V5.0** · Siemens PLCSIM Virtual Ethernet Adapter (Up). Nothing else to install.

Two things that differ from OpenPLC — read these first:

1. You **must** use **PLCSIM *Advanced***, not the regular PLCSIM. Only Advanced exposes a real TCP/IP endpoint on `:4840` that an external OPC UA client can reach.
2. A real/simulated Siemens server is **secured by default** and may not offer a `None` endpoint. For the **first** bring-up we deliberately **enable the "No security" policy** in TIA so we skip the certificate dance. We switch to secured (Basic256Sha256) in Part 8.

Plan (network numbers used throughout — change if they clash with your LAN):

- Virtual PLC IP: **`192.168.50.1`** / mask `255.255.255.0`
- Windows virtual-adapter IP: **`192.168.50.100`** / mask `255.255.255.0`
- OPC UA endpoint: **`opc.tcp://192.168.50.1:4840`**

Part 1 — Create the project + add an S7-1500 CPU

1. Open **TIA Portal V18** → **Create new project** → name it `CraneSim` → **Create**.
2. **Project view** (bottom-left "Project view" if you're in the Portal view).
3. In the project tree → double-click **Add new device** → **Controllers** → **SIMATIC S7-1500** → **CPU**. Pick a CPU that PLCSIM Advanced supports, e.g. **CPU 1511-1 PN** (`6ES7 511-1AK02-0AB0`) or **CPU 1516-3 PN/DP**. Choose the **latest firmware** offered (V2.9+). Click **OK**.

> Any S7-1500 works — all support the OPC UA server (firmware ≥ V2.0). If unsure, `1511-1 PN` is the lightweight default.

Part 2 — Set the CPU IP address

1. In **Device view**, click the **PROFINET port** (the green PN interface on the CPU).
2. **Properties** → **General** → **Ethernet addresses**.
3. Set **IP address =** **`192.168.50.1`**, **Subnet mask =** **`255.255.255.0`**. (No router needed.) Leave "Use IP protocol" on.

Part 3 — Enable the OPC UA server + (POC) allow "No security"

1. Click the **CPU** itself → **Properties** → **General** → **OPC UA** → **Server**.
2. Tick ✅ **Activate OPC UA server**. Note the shown endpoint: `opc.tcp://192.168.50.1:4840`.
3. Under **OPC UA** → **Server** → **Security** → **Security policies available**:
   - For **first bring-up**, tick ✅ **No security** (this makes the server advertise a `None` endpoint so we can connect without certificates).
   - Leave `Basic256Sha256` ticked too (we use it in Part 8).
4. Under **OPC UA** → **Server** → **Security** → **User authentication**:
   - Tick ✅ **Enable guest authentication** (anonymous). Good enough for the POC.
5. **Runtime license (compile requirement):** CPU → **Properties** → **General** → **Runtime licenses** → **OPC UA** → set **Purchased license = "SIMATIC OPC UA S7-1500 small"**. This only satisfies the TIA compiler — PLCSIM Advanced does **not** enforce a physical license.

> **Cert dance skipped for now** because we enabled *No security*. In Part 8 we turn it off and do the one-time trust + clock steps (the same gotcha as OpenPLC and real hardware).

Part 4 — Create the data (a global DB) our app will read/write

We need one tag the PLC writes (a counter we **monitor**) and one tag OPC UA writes (a bool we **command**).

1. Project tree → **Program blocks** → **Add new block** → **Data block (DB)** → name `PlcData` → type **Global DB** → **OK**. (Leave "Optimized block access" on — OPC UA uses symbolic names.)

2. Add these rows in the DB table:

Name	Data type	Start value	Purpose
`run`	`Bool`	`false`	**WRITE** (software → PLC command)
`cycle_count`	`Int`	`0`	**READ** (PLC → software monitor)
`reset`	`Bool`	`false`	**WRITE** (optional — zero the counter)
`lastClock`	`Bool`	`false`	internal edge-detect helper (leave alone)

3. For `run` and `reset`, in the DB table make sure the OPC UA columns are ticked (they are by default): **Accessible from HMI/OPC UA** ✅ and **Writable from HMI/OPC UA** ✅. Without *Writable*, our software command will be rejected.

Part 5 — Minimal program (make the counter move)

We increment `cycle_count` once per second while `run` is true — human-readable.

1. **Enable the 1 Hz system clock:** CPU → **Properties** → **General** → **System and clock memory** → **Clock memory** → tick ✅ **Enable the use of clock memory byte**. Leave the byte address at its **default** `MB0` (any byte works — the code uses the tag *name*, not the address). This creates the tag `Clock_1Hz` (bit `%M0.5` when the byte is `MB0`).

2. Open **OB1** (Main). Set its language to **SCL** (right-click OB1 → Properties, or pick SCL when adding). Paste this into the **code body**:

```
// Increment once per rising edge of the 1 Hz clock, only while running.
IF "PlcData".run AND "Clock_1Hz" AND NOT "PlcData".lastClock THEN
    "PlcData".cycle_count := "PlcData".cycle_count + 1;
END_IF;
"PlcData".lastClock := "Clock_1Hz";

// Optional: software can zero the counter by writing reset = TRUE.
IF "PlcData".reset THEN
    "PlcData".cycle_count := 0;
    "PlcData".reset := FALSE;   // auto-clear so it behaves like a pushbutton
END_IF;
```

3. **Compile:** right-click the CPU → **Compile** → **Hardware and software (only changes)**. Fix any red errors before continuing. (A yellow OPC-UA-license warning is fine.)

Part 6 — Start PLCSIM Advanced V5.0 in TCP/IP mode + download

This is the step that makes the OPC UA server reachable from Windows.

1. Launch **S7-PLCSIM Advanced V5.0** (the small control-panel app).
2. At the top, set **Online Access** to ✅ **PLCSIM Virtual Eth. Adapter** (NOT the plain "PLCSIM" softbus — that can't be reached by an external OPC UA client).
3. In **"TCP/IP communication with"**, select `Siemens PLCSIM Virtual Ethernet Adapter`.
4. Under **Start Virtual S7-1500 PLC**: type an **Instance name** e.g. `CraneSim`, leave the IP fields (they'll be set from the project on download), click **Start**. A row appears with a **yellow** icon (powered, not yet programmed).
5. **Give the Windows adapter a matching IP** (so our app can reach the PLC):
   - Windows **Settings** → **Network** → **Change adapter options** → right-click **"Ethernet ... Siemens PLCSIM Virtual Ethernet Adapter"** → **Properties** → **IPv4** → **Properties**.
   - Set **IP** `192.168.50.100`, **mask** `255.255.255.0`, no gateway. **OK**.
   - (One-time. See the PowerShell one-liner in the appendix to script this.)
6. Back in **TIA**: right-click the **CPU** → **Download to device**.
   - **Type of PG/PC interface** = **PN/IE**
   - **PG/PC interface** = **Siemens PLCSIM Virtual Ethernet Adapter**
   - Click **Start search** → the `CraneSim` instance appears → select it → **Load**.
   - Accept the dialogs → **Load** → **Finish** (choose **Start module** so the CPU goes RUN).
7. In the PLCSIM Advanced panel the instance icon turns **green (RUN)**. The OPC UA server is now live at `opc.tcp://192.168.50.1:4840`.

Quick reachability check (optional):

```
Test-NetConnection 192.168.50.1 -Port 4840   # TcpTestSucceeded : True
```

Part 7 — Find the exact NodeIds, then point our app at the server

Siemens NodeIds for DB tags look like: `ns=3;s="PlcData"."cycle_count"`. The namespace index is **usually 3** for the standard SIMATIC address space, but **verify it** — don't guess:

Option A — browse with UaExpert (recommended, 2 min): install the free Unified Automation *UaExpert* client, add server `opc.tcp://192.168.50.1:4840` (security *None*, *Anonymous*), browse **Objects** → **PlcData** → click `cycle_count` → read its **NodeId** from the Attributes pane.

Option B — TIA export: CPU → OPC UA → **Export OPC UA XML** and read the NodeIds from the file.

Once you have the real NodeIds, update **`backend/.env`** (I'll do this with you):

```
# --- Siemens S7-1500 (PLCSIM Advanced V5.0) — POC, No security ---
OPCUA_ENDPOINT=opc.tcp://192.168.50.1:4840
OPCUA_READ_NODE=ns=3;s="PlcData"."cycle_count"
OPCUA_WRITE_NODE=ns=3;s="PlcData"."run"
OPCUA_SECURITY_MODE=None
OPCUA_SECURITY_POLICY=None
OPCUA_PKI_DIR=
```

Then restart the backend and open the **PLC page**:

- **Monitor:** with `run = true`, `cycle_count` ticks up ~1/sec in the live poll. ✅ PLC → software.
- **Command:** toggle the write node (`run`) from the UI → the counter starts/stops. ✅ software → PLC.

> On the PLC page you can also type the NodeIds live (the endpoint/NodeIds/security are all runtime-selectable). The `.env` values are just the defaults it pre-fills.

Part 8 — Switch to secured (Basic256Sha256) — do this after the POC works

Real Siemens deployments are secured. To rehearse it in sim:

1. In TIA, CPU → OPC UA → Server → Security → **untick "No security"**, keep **Basic256Sha256**. Under User authentication you may keep guest or require a user.
2. Recompile + reload to PLCSIM Advanced.
3. Update `.env`:

   ```
   OPCUA_SECURITY_MODE=SignAndEncrypt
   OPCUA_SECURITY_POLICY=Basic256Sha256
   ```

4. **First connect will fail** — the server rejects our unknown client cert. This is the same two-step PLC-side gotcha we hit with OpenPLC / a real PLC:
   - **Trust our cert:** the Siemens server drops our client cert into a *rejected* store. Move it to *trusted*. In PLCSIM-Advanced sim the server's PKI is exposed via the TIA project's OPC UA security dialog /

the instance's certificate manager — accept/trust our client cert (`backend/pki/own/certs/client_certificate.pem`). Enabling **"Automatically accept client certificates during runtime"** in the TIA OPC UA security settings is the sim shortcut.
    - **Fix the clock:** OPC UA validates our cert against the **server** clock. PLCSIM Advanced uses the PC clock by default, so just make sure Windows time is correct — a wrong clock makes our cert look "not yet valid" and drops the secure channel even after trusting.
5. Reconnect → secured read/write should work identically.

Troubleshooting

Symptom	Cause / fix
`Test-NetConnection …:4840` fails	PLCSIM Advanced not in **Virtual Eth. Adapter** mode, instance not **green/RUN**, or Windows adapter IP not in `192.168.50.x`. Re-check Part 6.5.
`End point must exist … securityMode = None`	Server isn't advertising `None`. Either tick **No security** in TIA (Part 3.3) or set `.env` to the secured pair (Part 8). Same gotcha as a real PLC.
Connects but write rejected / `BadUserAccessDenied`	The DB tag isn't **Writable from HMI/OPC UA** (Part 4.3), or guest auth is off (Part 3.4).
`BadNodeIdUnknown`	Wrong NodeId. Browse with UaExpert (Part 7) — namespace index or the quoted `"DB"."tag"` casing is off.
Counter never moves	`run` is false, or clock memory byte not enabled (Part 5.1), or CPU in STOP.
Compile error on OPC UA	Set the **Runtime license** to "small" (Part 3.5).
`EADDRINUSE :4840`	Another OPC UA server (e.g. OpenPLC) is still bound to `:4840`. Only one server per port — stop the other.

Appendix — set the Windows virtual-adapter IP by script

```
# Find the adapter, then set a static IP in the PLC subnet (run as Admin).
$a = Get-NetAdapter | Where-Object { $_.InterfaceDescription -match 'PLCSIM
Virtual Ethernet' }
New-NetIPAddress -InterfaceIndex $a.ifIndex -IPAddress 192.168.50.100 -
PrefixLength 24 -ErrorAction SilentlyContinue
```

Deploying to the real hardware — note on S7-1200 (our actual target)

We **simulate with an S7-1500** because **PLCSIM Advanced only simulates S7-1500** (not the S7-1200) — and PLCSIM Advanced is the only simulator that exposes a reachable OPC UA endpoint. The simulated 1500 is a faithful stand-in: for our use (read a tag = monitor, write a tag = command) an S7-1200 and S7-1500 OPC UA server look the same to the client. **Moving to the real S7-1200 stays a `.env`-only change** (new endpoint IP + NodeIds). What differs on the 1200:

- **Firmware ≥ V4.4 required** — that's when Siemens added the OPC UA **server** to the S7-1200. Older firmware has no OPC UA at all.
- **License:** needs the **SIMATIC OPC UA S7-1200** runtime license ("small" tier only).
- **Server-only, tag read/write** — no methods, no companion-spec server interfaces, and the 1200 can't act as an OPC UA *client*. None of that affects us — our app is the client and only reads/writes tags.
- **NodeIds** use the same `ns=…;s="DB"."tag"` format, so the app config carries straight over.
- Enable the server the same way: device config → **OPC UA** → **Server** → **Activate** (+ security, guest auth, and the runtime license), then point `.env` at `opc.tcp://<1200-ip>:4840`.

What maps from OpenPLC → Siemens (so it clicks)

Concept	OpenPLC v4	Siemens S7-1500 (this runbook)
OPC UA server	plugin (`plugins.conf`, resets each shutdown)	built into the CPU firmware, enabled in device config
Simulator	the runtime itself	**PLCSIM Advanced V5.0** (Virtual Eth. Adapter)
Tag → NodeId	`opcua.json` `index` from `debug.c`	symbolic `ns=3;s="DB"."tag"`
Write role	give `viewer` `"rw"`	tag **Writable from HMI/OPC UA** + guest auth
Security	POC server = `None` only	secured by default; we enable `None` for POC, then Basic256Sha256
Our app	no change	**no change** — `.env` only